

Control de Velocidad de Motor DC

Hernández Alexis de la Cruz, Jorja Fernando Javier

UTN FRA - Técnicas Digitales III

Avellaneda, Argentina

alexisdelacruzhernandez@gmail.com

nuevacuentafernando@gmail.com

Abstract— El proyecto realizado consiste en el control de velocidad de un motor DC mediante la aplicación del control PID, el mismo cuenta con ajustes de velocidad final, sentido de giro, tiempo de aceleración y tiempo de desaceleración. Implementa, además, una función de datalogger con un reloj en tiempo real (RTC) y una calibración previa de los parámetros del control PID. Ha sido desarrollado con un microcontrolador RP2350 (arquitectura Cortex-M33), stick Raspberry Pi Pico 2W, utilizando un sistema operativo en tiempo real (FreeRTOS). Se programó en lenguaje C con el entorno de desarrollo Visual Studio Code v1.102.3 con la extensión Raspberry Pi Pico v0.18.0.

I. ÍNDICE

I. ÍNDICE.....	1
II. FUNDAMENTACIÓN.....	1
A. Problemática.....	1
B. Fundamentación Teórica de Hardware.....	1
C. Fundamentación Teórica de Software.....	1
III. ALCANCE.....	2
IV. DIAGRAMA EN BLOQUES.....	2
V. SOBRE EL HARDWARE.....	2
A. Stick Raspberry Pi Pico 2W.....	2
B. Puente H L298N.....	2
C. Display LCD 16x2.....	2
D. Real Time Clock (RTC) DS1307.....	3
E. Módulo SD.....	3
F. Sensor Infrarrojo.....	3
G. Encoder Rotativo.....	3
H. LEDs.....	3
I. Pulsadores.....	3
J. Motor.....	4
VI. SOBRE EL SOFTWARE.....	4
A. Tareas.....	4
B. Medición de Velocidad.....	4
C. Control PID.....	4
D. Autoajuste.....	4
E. Datalogger.....	4
V. PCB.....	4
A. Esquemático.....	4
B. Diseño de Placa.....	4
C. Test Points.....	5
VI. CONCLUSIONES.....	5
VII. REFERENCIAS.....	5
VIII. ANEXOS.....	6

II. FUNDAMENTACIÓN

A. Problemática

En muchas aplicaciones se hace necesario implementar un sistema de control de velocidad para motores, con el objetivo de mantener una velocidad constante frente a perturbaciones externas y/o seguir curvas específicas de aceleración y desaceleración. Esto permite mejorar la precisión del sistema y optimizar el consumo energético.

B. Fundamentación Teórica de Hardware

Para alcanzar el objetivo principal de control, es necesario muestrear la velocidad del motor, a fin de contrastar con el valor de referencia definido. Para ello, se empleó un sensor infrarrojo de herradura (WYC H206), en conjunto con una rueda dentada acoplada al eje del motor. Durante el giro, los dientes de la rueda interrumpen y restablecen periódicamente el haz de luz infrarroja, generando pulsos que pueden ser contados y utilizados para calcular la velocidad angular del eje.

Se seleccionó el puente H L298N[1] como actuador, el cual permite controlar tanto el sentido de giro como la velocidad del motor. El cambio de sentido se logra mediante la activación selectiva de los transistores internos, mientras que la velocidad se regula aplicando una señal PWM en la habilitación del mismo.

La información relevante del sistema es visualizada a través de una pantalla LCD 16x2. La interfaz de usuario permite el ajuste de parámetros mediante un encoder rotativo y dos pulsadores.

Para la implementación datalogger, se utiliza un RTC, que proporciona la referencia temporal necesaria para el etiquetado cronológico. Los datos se almacenan en una memoria microSD, organizada en dos archivos: un archivo .txt, donde se guardan las constantes del control PID, y un archivo .csv que registra la fecha, hora, velocidad, frecuencia del sensor, ancho de pulso del PWM y sentido de giro.

C. Fundamentación Teórica de Software

El proyecto ha sido desarrollado sobre un sistema operativo en tiempo real, específicamente FreeRTOS[2], lo cual implica la utilización de tareas concurrentes coordinadas mediante mecanismos de sincronización como colas y semáforos.

Para la obtención de la velocidad del motor, se implementó una interrupción de GPIO que detecta los flancos generados por el sensor infrarrojo. Estos eventos son gestionados mediante un semáforo del tipo contador, y una tarea dedicada se encarga de leer la cantidad de pulsos acumulados, calcular la velocidad angular correspondiente y enviarla a través de una cola.

La pantalla LCD y el RTC se comunican mediante el protocolo I²C. Dado que ambos dispositivos comparten el mismo bus (recurso), se hace necesario asegurar la exclusión mutua durante su acceso. Para ello, se emplea un semáforo del tipo mutex, que garantiza que solo una tarea pueda utilizar el bus I²C a la vez,

La comunicación con la memoria SD utiliza el protocolo SPI, como no es un recurso compartido no se requiere exclusión mutua.

III. ALCANCE

- Medir la velocidad del motor y calcular la frecuencia generada por el sensor infrarrojo, con una resolución de $\pm 10\text{RPM}$ y una actualización cada 120ms.
- Implementar un control PID que permita al motor seguir la velocidad establecida como set point.
- El set point de velocidad sigue rampas de aceleración y desaceleración en función de los tiempos configurados por el usuario.
- Indicar visualmente el sentido de giro del motor.
- Configurar, mediante un encoder rotativo y dos pulsadores las variables del sistema: velocidad final, tiempo de aceleración, tiempo de desaceleración y la hora del RTC.
- Calibrar las constantes del control PID al inicio del programa.
- Almacenar en una memoria microSD las constantes del control PID en un archivo de texto .txt, y registrar en un archivo .csv la fecha, hora, velocidad del motor, frecuencia del sensor, ancho de pulso PWM y sentido de giro.

IV. DIAGRAMA EN BLOQUES

Los módulos que componen el sistema se muestran en la figura 1, donde se pueden ver las conexiones entre ellos y a que pines corresponden.

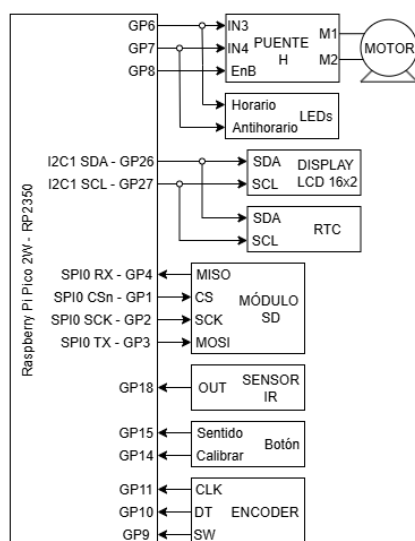


Fig. 1 Diagrama en bloques del control de velocidad

V. SOBRE EL HARDWARE

A. Stick Raspberry Pi Pico 2W

El pinout del stick se muestra en la figura 2. Las características principales utilizadas en el proyecto son:

- Microcontrolador RP2350[3] (Cortex-M33).
- Frecuencia de funcionamiento de hasta 150MHz.
- Tensión de alimentación de 5V, regulada internamente a 3,3V para el RP2350.
- 26 GPIO disponibles.
- 2 controladores SPI.
- 2 controladores I²C.
- 24 canales PWM.

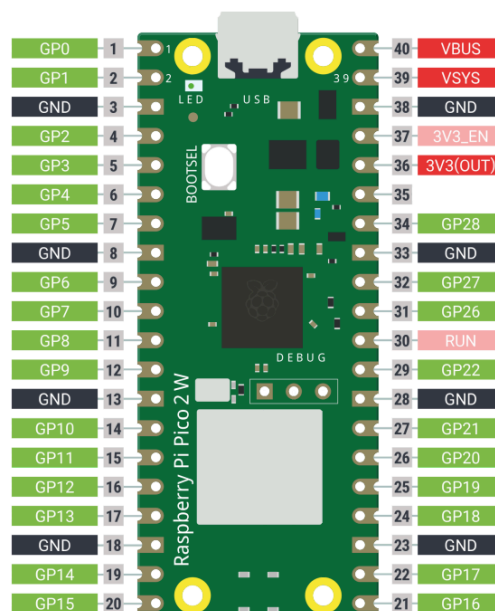


Fig. 2 Pinout stick Raspberry Pi Pico 2W

B. Puente H L298N

Este integrado cuenta con 2 puentes H integrados, donde uno se usó para controlar al motor. El pinout se encuentra en la figura 3. Las características principales del integrado son:

- Tensión de alimentación de 46V máximo.
- Tensión de alimentación lógica de 7V máximo.
- Corriente de salida:
 - No repetitiva: 3A.
 - Repetitiva: 2,5A.
 - Continua: 2A.

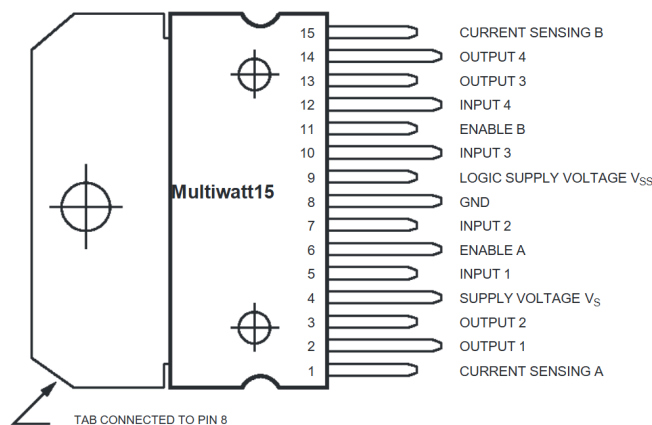


Fig. 3 Pinout L298N

C. Display LCD 16x2

Se utilizó junto al adaptador I²C PCF8574T[4]. El pinout se muestra en la figura 4. Sus características principales son:

- Tensión de alimentación 6V máximo.
- Frecuencia máxima de I²C de 100kHz.
- Cuenta con dos líneas de 16 caracteres.
- Consumo de corriente máximo de 3mA.

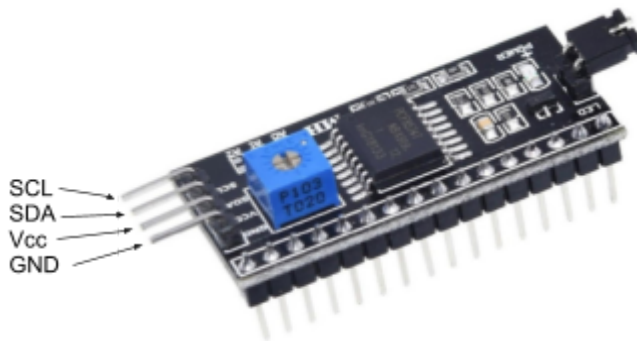


Fig. 4 Pinout adaptador I²C

D. Real Time Clock (RTC) DS1307

Para guardar registro temporal de las muestras se optó por utilizar el RTC DS1307[5], mostrado en la figura 5. Sus características principales son:

- Tensión de alimentación 7V máximo.
- Tensión de batería de 3,5V máximo.
- Frecuencia máxima de I²C de 100kHz.
- Guarda registros de año, mes, día, día de la semana, hora, minuto y segundo.

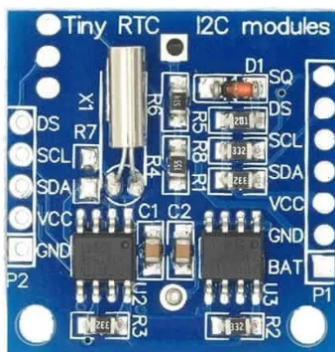


Fig. 5 Pinout módulo DS1307

E. Módulo SD

Con el fin de exportar los datos para visualizar en una PC se optó por grabarlos en una tarjeta micro SD mediante el módulo microSD de la figura 6. Las características usadas fueron:

- Interfaz SPI a 1MHz.
- Tensión de alimentación de 5V, regulada a 3,3V por módulo.
- Soporte para formatos FAT.



Fig. 6 Pinout módulo microSD

F. Sensor Infrarrojo

Como se mencionó en la sección II apartado B, para medir la velocidad se empleó un sensor infrarrojo de herradura, el mismo se muestra en la figura 7.

- Permite alimentación de 3,3V y 5V.
- Salida normalmente abierta, si no está bloqueado a la salida tiene un 1 y si está bloqueado tiene un 0.
- Ranura de 5mm.

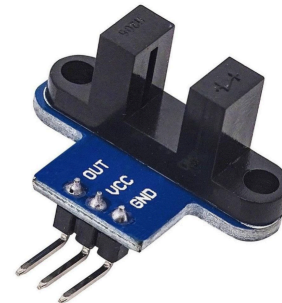


Fig. 7 Pinout módulo sensor infrarrojo

G. Encoder Rotativo

Para interactuar con los menús se optó por usar un encoder rotativo KY-040 mostrado en la figura 8. Sus características principales son:

- Cuenta con dos salidas en cuadratura para detectar paso y sentido.
- 20 pulsos por revolución.
- Cuenta con interruptor.

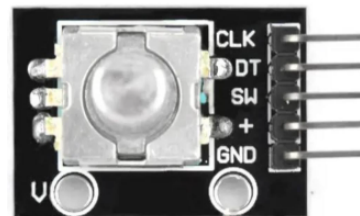


Fig. 8 Encoder rotativo KY-040

H. LEDs

Se integraron en el PCB dos LEDs, uno verde y uno rojo para indicar el sentido de giro, cuando el LED verde está encendido gira en sentido horario, el LED rojo indica giro en sentido antihorario, si ninguno está encendido el motor está parado.

I. Pulsadores

Se emplearon dos botones, además del integrado en el encoder, para facilitar la interacción del usuario.

El botón del encoder al ser pulsado inicia el control hacia la nueva velocidad, al ser mantenido se cambia de menú para configurar los tiempos de aceleración y/o desaceleración.

El botón "Calibrar" se emplea al inicio del programa para configurar la hora del RTC y comenzar el ajuste de los parámetros del PID.

El botón "Sentido" al ser presionado cambia el sentido de la velocidad objetivo, al ser mantenido fija el objetivo en 0RPM.

J. Motor

Como planta del sistema se empleó un motor genérico de impresora reciclado, el mismo fue utilizado a 12V y cuenta con aproximadamente unas 3000RPM maximas.

VI. SOBRE EL SOFTWARE

A. Tareas

El software está formado por 11 tareas, donde 3 de ellas se eliminan al terminar la calibración, el diagrama de tareas que esquematiza la interacción entre cada una de ellas se encuentra en el Anexo 1 - Diagrama de tareas.

Contamos con una tarea central que recibe la velocidad medida, el giro del encoder, la pulsación del encoder y del botón "Sentido", en base al evento sucedido actualiza una cola con la configuración realizada.

La cola de configuración es leída a su vez por la tarea de control del motor, la del display LCD y la del datalogger, donde estas se desbloquean cada determinado tiempo fijado.

Inicialmente se crea una tarea para configurar la hora del RTC, donde también es necesaria la tarea del encoder y del pulsador calibrar, una vez configurado se crea la tarea de tuning del PID, junto a la que lee la velocidad, la tarea de control y la tarea central.

Una vez finalizada la calibración se crean el resto de tareas y comienza el flujo del programa principal.

B. Medición de Velocidad

Como se mencionó anteriormente, el sensor infrarrojo se lee mediante una interrupción de doble flanco que incrementa un contador, en base a los pulsos leídos, el número de ranuras y el tiempo de muestreo se calcula la velocidad mediante la ecuación (1).

$$\text{Velocidad} = \frac{\text{Pulsos}}{\text{Ranuras}} \cdot \frac{60000}{T_{\text{MUESTREO}}} \quad (1)$$

Se diseñó e imprimió en 3D una rueda dentada de 25 ranuras, de forma tal que efectivamente tenemos 50 cuentas por revolución, para contar con una resolución de $\pm 10\text{RPM}$ se necesitó un tiempo de muestreo de 120ms.

C. Control PID

Para implementar el control PID se utilizó la ecuación (2).

$$y_n = y_{n-1} + \left(kp + \frac{kd}{tm}\right) \cdot e_n + \left(-kp + ki \cdot tm - \frac{2 \cdot kd}{tm}\right) \cdot e_{n-1} + \frac{kd}{tm} \cdot e_{n-2} \quad (2)$$

Inicialmente se encontró mediante prueba y error los valores de kp, kd y ki, siendo tm el tiempo que tarda en aplicarse nuevamente la fórmula, en este caso 40ms.

Los valores obtenidos fueron:

- $kp = 0,5$
- $ki = 0,6$
- $kd = 0,005$

La salida obtenida se debe encontrar en el rango de 0 a 2000 ya que estas son las cuentas del PWM, además, debemos tomar en cuenta que a el motor comienza a moverse a partir de 350 cuentas aproximadamente, para solucionar este problema levantamos el nivel mínimo de pwm a 350 cuentas mediante lo que se indica en la ecuación (3).

$$y_n = 350 + 0,825 \cdot y_n \quad (3)$$

Finalmente aplicamos la saturación del controlador, si y_n es mayor a 2000 lo mantenemos en 2000 y si es menor a 360 lo mandamos a 0, de forma tal de asegurar el frenado del motor.

D. Autoajuste

Por limitaciones físicas y de medición no fue posible implementar el método de Ziegler-Nichols, para dar una solución se optó por hacer un barrido de constantes kp, ki y kd alrededor de las constantes obtenidas manualmente, de forma tal de quedarnos con la que nos de menor error total frente a la respuesta al escalón.

Los valores que se ponen a prueba son:

- $kp = 0,4 - 0,5 - 0,6$
- $ki = 0,6 - 0,7 - 0,8 - 0,9$
- $kd = 0,004 - 0,005 - 0,006$

E. Datalogger

A continuación se enumeran las consideraciones que se tuvieron en cuenta para el correcto guardado de datos en la tarjeta SD:

- Verificación del correcto montaje de la SD
- Verificación de espacio y nombre disponible para crear el nuevo archivo de datos .csv
- Guardado cada 5 muestras, de forma que se guarde la información cada 5 segundos, minimizando la posibilidad de extraer la tarjeta durante la escritura.
- Detección de desconexión.
- Detección de reconexión de la misma SD.

Para implementarlo se optó por utilizar la librería FatFs[6], permitiéndonos la gestión de archivos y de la SD.

V. PCB

A. Esquemático

El esquemático completo se muestra en el Anexo 2 - Esquemático, el mismo fue desarrollado en KiCad 9, siendo el mismo una única placa con todo integrado, alimentada con 12V y regulada a 5V para la alimentación.

B. Diseño de Placa

En la figura 9 se muestra el diseño de las pistas, mientras que en la figura 10 se muestra la distribución de los componentes utilizados. En el Anexo 3 - PCB se detalla.

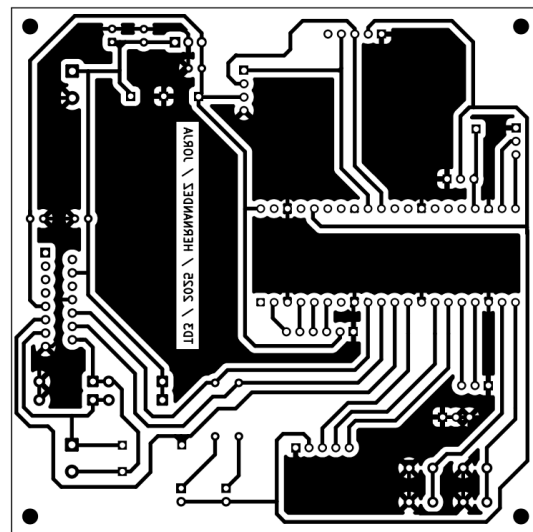


Fig. 9 Vista de las pistas del PCB

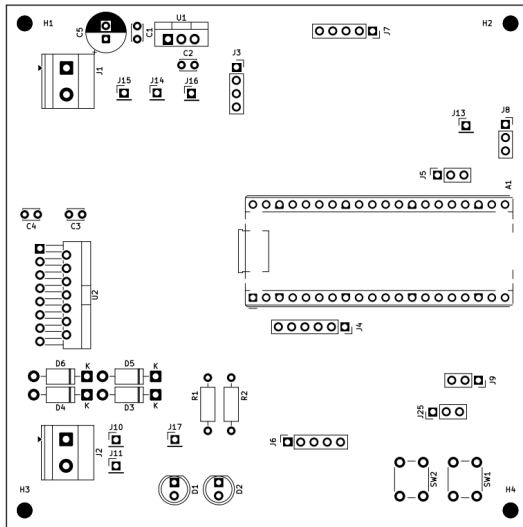


Fig. 10 Vista de componentes del PCB

C. Test Points

Para verificar el correcto funcionamiento del sistema se optó por tener los siguientes test points, indicados también en la figura 11:

- Sensor infrarrojo: se mide sobre su salida para dar cuenta de su frecuencia, pudiendo contrastar con la medida del proyecto realizado.
- L298N: se mide sobre el Enable del canal B del mismo, donde se aplica el PWM de control para el motor. No se mide sobre IN3 e IN4 ya que las mismas tienen conectados LEDs.
- Motor: se mide sobre ambas salidas del puente H para dar una idea de la tensión equivalente sobre el motor.
- Alimentación: se miden los 12V y los 5V para asegurar una correcta alimentación.

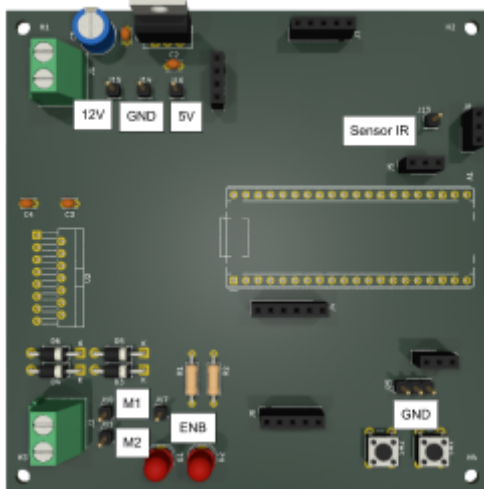


Fig. 11 Test points utilizados

VI. CONCLUSIONES

Si bien en el alcance de velocidad no indicamos velocidades mínimas y máximas nos encontramos con limitaciones físicas de hardware que limitan estas:

- Velocidad mínima: al ensayar al motor obtuvimos una velocidad mínima de funcionamiento de unas 200RPM, siendo esta muy inestable, por lo que se eligió limitar el

rango de funcionamiento a un mínimo de 500RPM.

- Velocidad máxima: al medir el motor en vacío y sin puente H obtuvimos las 3000RPM mencionadas anteriormente, al colocar el puente H y realizar nuevamente las mediciones observamos un máximo de aproximadamente 2700RPM, por lo que fijamos el máximo configurable a esta velocidad.
- Frecuencia de PWM: si bien no es mencionado anteriormente, al realizar ensayos sobre el motor encontramos que para una frecuencia de 500Hz el motor se comienza a mover con las 350 cuentas mencionadas. Si se busca aumentar la frecuencia lo que se obtiene es un mayor porcentaje de ciclo de actividad necesario para comenzar el movimiento, junto a una velocidad mínima mayor.

Con respecto al seguimiento de las rampas de aceleración y desaceleración podemos decir que las sigue adecuadamente excepto cuando la velocidad objetivo es cercana al límite máximo, esto se solucionaría aumentando el valor de k_p , pero esto mismo produce mayor error en el seguimiento de las rampas, por lo que se decidió seguir con mayor precisión las rampas pero tener el inconveniente mencionado para altas velocidades.

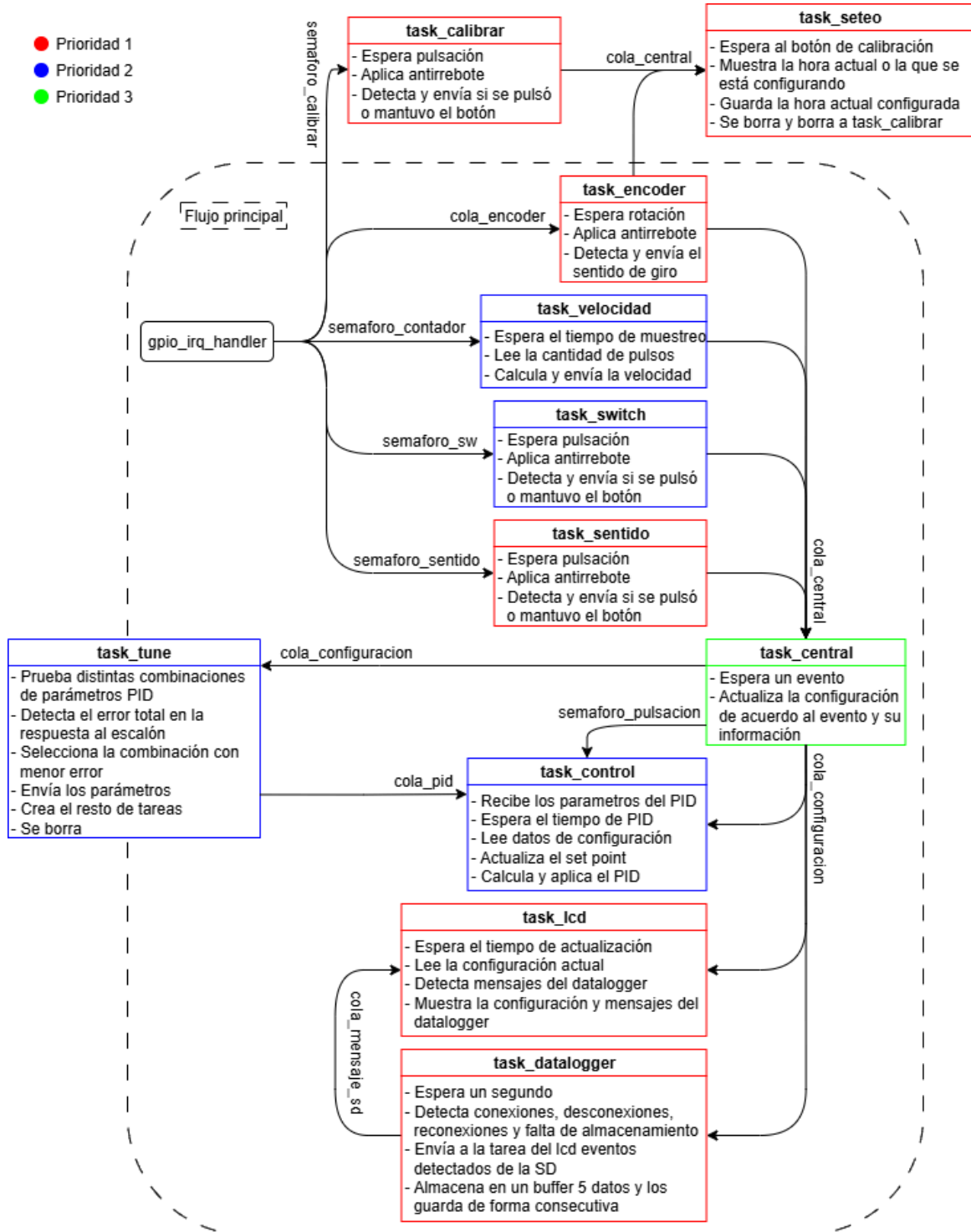
Nos encontramos con un problema a la hora de implementar el control PID, y es que el tiempo de muestreo para obtener una precisión aceptable en la medición de velocidad es muy largo para implementar ajustes rápidos sobre el control PID, para solucionar esto optamos por realizar 3 cálculos de PID por cada velocidad medida, simplemente cambiando el valor del setpoint, permitiéndonos conseguir un mejor control sobre el motor.

VII. REFERENCIAS

- [1] L298N
<https://www.st.com/resource/en/datasheet/l298.pdf>
- [2] FreeRTOS
<https://www.freertos.org>
- [3] RP2350
<https://datasheets.raspberrypi.com/rp2350/rp2350-datasheet.pdf>
- [4] PCF8574
<https://www.ti.com/lit/ds/symlink/pcf8574.pdf>
- [5] DS1307
<https://www.analog.com/media/en/technical-documentation/data-sheets/ds1307.pdf>
- [6] FatFs
<https://elm-chan.org/fsw/ff/>
- [7] Video del proyecto funcionando
<https://drive.google.com/file/d/12mVOL6ROZHUq80TI8ynDZo54uyhTF7jL/view?usp=drivesdk>

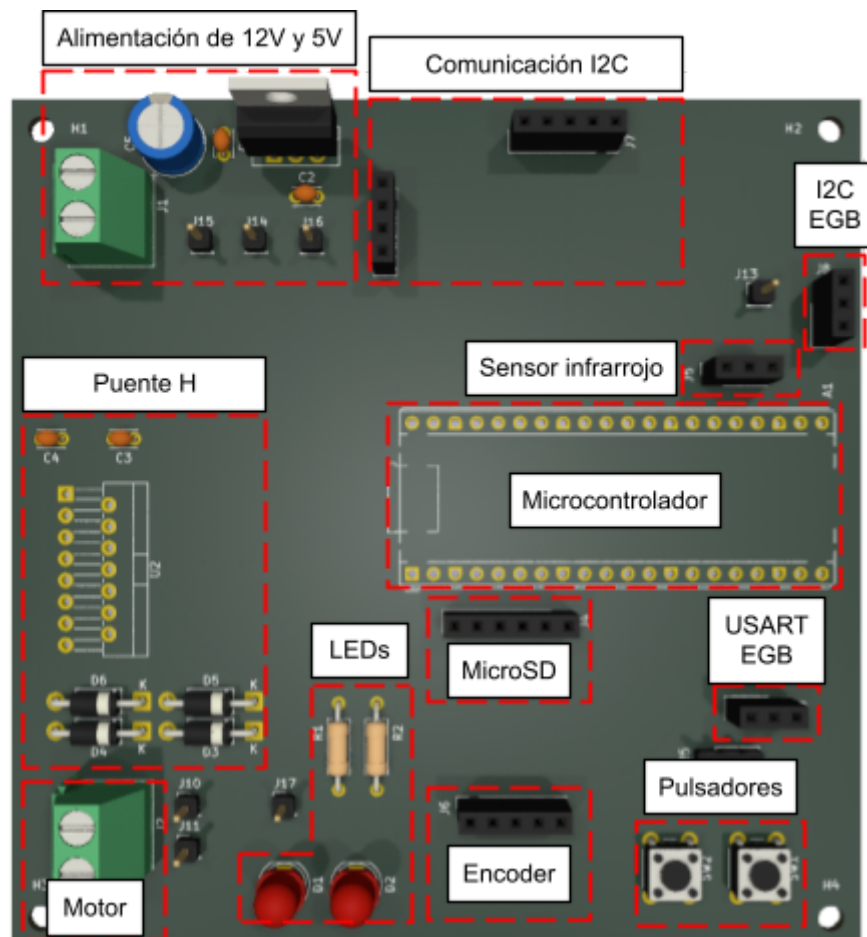
VIII. ANEXOS

ANEXO 1 - Diagrama de Tareas



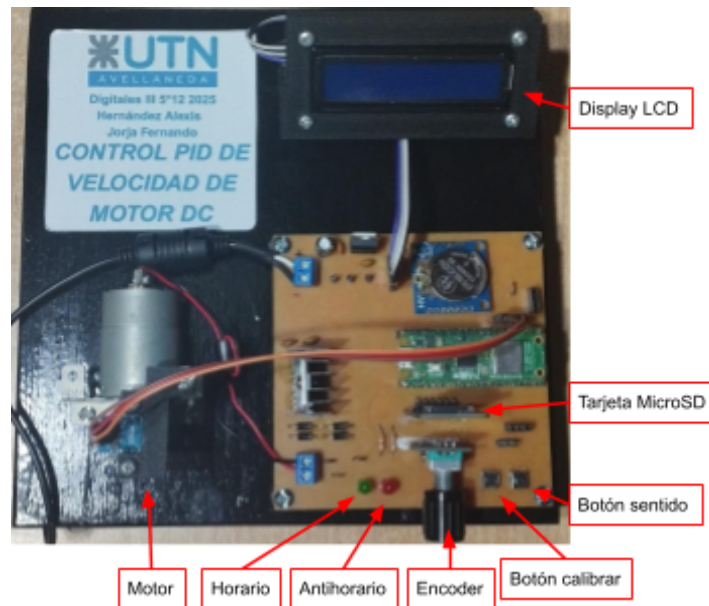
Página 7 de 11





ANEXO 4 - Guía de Uso

A continuación se muestra el ensamblado del proyecto, en la misma se señalan las partes que contiene, luego se desarrollan las funciones y cómo interactuar con cada una de ellas.



Al iniciar el sistema se mostrará en el display LCD la hora configurada anteriormente, en el formato DD/MM/AA HH:MM:SS, en caso de querer configurarla se deben seguir los siguientes pasos:

- Pulsar el botón calibrar, esto frenará el conteo y comenzará la configuración.
- Al rotar el encoder se modificará el valor seleccionado, el primer valor a configurar es el día de la fecha.
- Una vez seleccionado el día se debe pulsar nuevamente el botón calibrar, pasando a modificar el mes, se debe continuar de esta forma hasta completar la configuración.
- Cuando hayamos terminado la configuración se debe mantener pulsado el botón calibrar, esto nos llevará al ajuste de parámetros PID del sistema.
- Aparecerá en pantalla la palabra “Calibrando” y se debe esperar a que se detenga para comenzar con el flujo normal del programa.

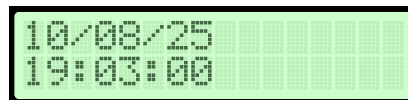


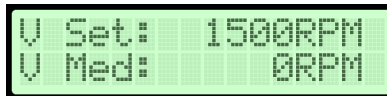
Imagen visualizada al iniciar



Mensaje mostrado mientras ajusta el PID

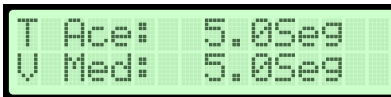
Una vez finalizada la inicialización se mostrará en pantalla un menú, mostrando la velocidad objetivo y por debajo la velocidad medida, ambas en RPM. A continuación se explicará cómo modificar las variables del sistema:

- Al girar el encoder en este menú se modificará la velocidad objetivo.
- Si se mantiene pulsado el botón del encoder se pasará al segundo menú, donde se muestra el tiempo de aceleración y el tiempo de desaceleración.
- Al girar el encoder se modificará el tiempo de aceleración, si se desea modificar el tiempo de desaceleración se deberá pulsar el encoder una vez, pasando a modificarlo.
- Para volver al menú anterior se debe mantener nuevamente el botón del encoder.
- Para comenzar el movimiento del motor hacia la velocidad deseada se debe pulsar una vez el encoder, la velocidad del motor variará linealmente hasta la velocidad objetivo en el tiempo de aceleración o desaceleración (dependiendo si el objetivo es mayor o menor a la velocidad actual) configurado anteriormente.



U Set: 1500RPM
U Med: 0RPM

Menú de configuración de velocidad



T Ace: 5.0Seg
U Med: 5.0Seg

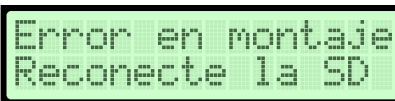
Menú de configuración de tiempos

En caso de querer variar rápidamente la velocidad objetivo se cuenta con el botón sentido:

- Al pulsarlo una vez se cambiará el sentido de la velocidad objetivo (cambio de signo).
- Al mantenerlo pulsado se fijará la velocidad objetivo en cero.

Finalmente se detallarán las características del sistema de almacenamiento de la tarjeta MicroSD:

- Si ocurre un error en la lectura de la SD o si no está presente se mostrará en pantalla el mensaje “Error en montaje. Reconecte la SD”.



Error en montaje
Reconecte la SD

- Al ser leída correctamente pueden mostrarse dos mensajes:
 - “Almacenamiento lleno”, indicando que no hay espacio disponible para crear un nuevo archivo.



Almacenamiento
lleno

- “Archivo creado exitosamente”, indicando la correcta creación de los archivos.



Archivo creado
exitosamente

- Cuando ya se crearon los archivos y se desconecta la SD se mostrará “SD desconectada”.



SD desconectada

- Si se reconecta la misma MicroSD se mostrará “SD reconectada”, si es una MicroSD distinta se intentará crear nuevamente los archivos, indicando los mensajes mencionados anteriormente.



SD reconectada

Los archivos creados se llaman y contienen los siguientes datos:

- pid.txt: en su interior se guarda en cada línea los valores de k_p , k_i y k_d por cada encendido, siendo estos valores los que menor error obtuvieron en la calibración.
- dataXXXX.csv: por cada inicialización se crea un archivo distinto, pudiendo ser desde 0000 hasta 9999, en el mismo se almacena la fecha, la hora, la velocidad, la frecuencia del sensor, el ancho del pulso PWM y el sentido de rotación, siendo este P “parado”, H “horario” o A “antihorario”.

```
kp = 0.4; ki = 0.6; kd = 0.004  
kp = 0.4; ki = 0.6; kd = 0.004
```

Ejemplo de datos guardados en pid.txt

Fecha	Hora	Vel	Frec	DC	Sentido
10/8/2025	16:24:10	0	0.0	0.0	P
10/8/2025	16:24:11	0	0.0	0.0	P
10/8/2025	16:24:12	0	0.0	0.0	P
10/8/2025	16:24:13	0	0.0	0.0	P
10/8/2025	16:24:14	0	0.0	0.0	P
10/8/2025	16:24:15	0	0.0	0.0	P
10/8/2025	16:24:16	20	8.3	19.5	H
10/8/2025	16:24:17	320	133.3	21.7	H
10/8/2025	16:24:18	610	254.2	24.4	H
10/8/2025	16:24:19	880	366.7	28.5	H
10/8/2025	16:24:20	1170	487.5	31.9	H
10/8/2025	16:24:21	1380	575.0	35.0	H
10/8/2025	16:24:22	1460	608.3	35.6	H
10/8/2025	16:24:23	1490	620.8	35.8	H
10/8/2025	16:24:24	1500	625.0	35.8	H
10/8/2025	16:24:25	1510	629.2	35.7	H
10/8/2025	16:24:26	1490	620.8	36.0	H

Ejemplo de datos guardados en dataXXXX.csv